Experiment6.1

StudentName: Arushi Rana
UID:22BCS12899
Branch:BE-CSE
Section/Group: IoT_641(A)
Semester:6th
DateofPerformance:4/3/25
SubjectName: AdvancedProgramming-2
SubjectCode:22CSP-351

1. **Aim:** Check if a binary tree is a mirror of itself. Use recursive DFS or iterative BFS (queue) to compare corresponding nodes. The solution runs in O(n) time andO(h)space.Edgecasesincludeanemptytree(symmetric)andasingle-node tree.
2. **Objective:**Implementanalgorithmto checkiftreeissymmetrical.
3. **Implementation/Code:**

```
classSolution{ public:
    boolisSymmetric(TreeNode*root){ if
        (!root) return true;
        returnisMirror(root->left,root->right);
    }

    boolisMirror(TreeNode*t1,TreeNode*t2){ if
        (!t1 && !t2) return true;
        if (!t1 || !t2) return false;
        return(t1->val==t2->val)&&
               isMirror(t1->left,t2-
               >right)&&isMirror(t1->right, t2-
               >left);
    }
};
```

4. **Output**

Case1:
Input:
Root=[[1,2,2,3,4,4,3]
Output:
True

Case 2:
Input:
Root=[1,2,2,null,3,null,3]
Output:
false

Experiment6.2

1. **Aim**: - Binary Tree Inorder Traversal Return the inorder (left-root-right) traversalofabinarytreeusing recursionoraniterativestack-basedapproach. The solution runs in O(n) time and O(h) space. Edge cases include an empty tree ([]) and a left-skewed or right-skewed tree
2. **Objective**:Implementalgorithmstoefficientlyperform Binarytreeinorder traversal.

3. **Code**:

```
classSolution{ public:
    vector<int>inorderTraversal(TreeNode*root){
        vector<int> res;
        inorder(root,res);
        return res;
    }

private:
    voidinorder(TreeNode*node,vector<int>&res){ if
        (!node) {
            return;
        }
        inorder(node->left,  res);
        res.push_back(node->val);
        inorder(node->right,res);
    }
};
```

4. **Output**

```
Case1:
Input:
Root=[1,null,2,3]
[1,2,3,4,5,null,8,null,null,6,7,9]
Output:
[1,3,2]
```

```
Case2:
Input:
Root=

Output:
[4,2,6,5,7,1,3,9,8]
```

Experiment6.3

1. **Aim**: Find the kth smallest element in a BST using an in-order traversal (iterativeorrecursive).TheapproachrunsinO(k)timeandO(h)space.Edge cases include k = 1 (smallest element) and k = n (largest element).
2. **Objective**:Implementanalgorithmtofindthekthsmallestelementina BST.

3. **Code**:

```
classSolution{
    private:
        voidinorder(TreeNode*root,vector<int>&a){
            if(root==nullptr){
                return;
            }

            //traverseleftsubtree
            inorder(root->left, a);
            //storevalueofrootonwhichwearestanding
            a.push_back(root->val);
            //processtherightsubtree
            inorder(root->right,a);
        }

public:
    intkthSmallest(TreeNode*root,intk){
        //creatingavectorofint
        vector<int>a;
        //callinginordertraversal
        inorder(root, a);
        //returningtheans..
        return a[k-1];
    }
};
```

4. **Output**

Case1:
Input:
Root=[3,1,4,null,2]k=1
Output:
1

Case2:
Input:
root=[5,3,6,2,4,null,null,1]k=3
Output:
3

5. LearningOutcomes:

- Understandthestepsinvolvedin BFSandDFS.
- Learnttheconceptoftreesymmetry.
- Learnhowtoanalyzeandcomparethetimecomplexity.
- Implementappropriatealgorithmbasedontheproblem'sconstraints.